

Environmental Conditioning of Protein Sequence Generation from GOLD, UniProt, NASA POWER, and WorldClim

AI in Genomics Project

Abstract

This project asks whether environmental measurements attached to biological sampling locations can be used to condition a protein sequence generator. The implemented code builds a dataset by linking GOLD organism metadata to reviewed UniProt protein sequences, enriching sample coordinates with NASA POWER daily weather and WorldClim monthly climate rasters, cleaning invalid numerical sentinels, and training a recurrent neural network that generates amino-acid sequences from environmental inputs. The current model should be treated as a proof of concept: it demonstrates an end-to-end reproducible pipeline and an interactive Streamlit interface, but it does not yet prove that generated sequences are biologically functional or causally adapted to the environment.

1 Background

Proteins are chains of amino acids that fold into molecular machines. Their sequences are constrained by evolution, organism history, and environment. For example, proteins from organisms living in hot or dry habitats may be under different selective pressures than proteins from temperate habitats. Our goal is not to model all of evolution, but to test a narrower machine-learning question: can a model learn a relationship between coarse environmental variables and the distribution of protein sequences observed in organisms sampled from those environments?

The project combines four public resources. GOLD provides organism-level sample metadata, including taxonomy IDs and geographic coordinates when available [1]. UniProt provides reviewed protein sequences for each taxonomy ID [2]. NASA POWER provides daily surface temperature and radiation for the sample location/date [3]. WorldClim provides long-term monthly climate rasters, including precipitation, solar radiation, average temperature, and vapour pressure [4]. SoilGrids querying is implemented in the code, but the current merged training table does not include the returned pH or soil-carbon fields as model inputs [5].

2 Motivation

Most protein generation models are conditioned on sequence context, structure, or textual prompts. We explored a different conditioning signal: the physical environment associated with the organism from which a protein was sampled. If such a signal is useful, it could eventually help generate candidate protein sequences for unusual environmental settings. The current project tests the infrastructure and modeling feasibility of this idea rather than claiming a validated biological design tool.

3 Data

3.1 Source Data and Linking

The pipeline begins with the `Organism` sheet in `goldData.xlsx`. The loader caches this sheet as `goldData_Organism.parquet`. In the local cached copy, GOLD contains 530,160 organism rows; 64,665 rows have both latitude and longitude. The code sorts these coordinate-bearing rows and queries UniProt using each row’s NCBI taxonomy ID.

For each taxonomy ID, `src/main.py` calls the UniProt REST search API for up to five reviewed protein entries. The current query is `taxonomy_id:<id> AND reviewed:true`; it retrieves accession, entry name, organism name, and sequence. These protein rows are then joined back to GOLD by taxonomy ID and enriched by geographic location.

3.2 Environmental Covariates

Each unique latitude, longitude, and collection-date triple is enriched with:

- NASA POWER daily temperature at 2 meters and shortwave radiation. These are stored as `NASA_Temp_C` and `NASA_Radiation`. If GOLD lacks a parseable collection date, the code uses 2015-01-01 as a fallback date.
- WorldClim monthly precipitation (`prec_01–prec_12`), solar radiation (`srad_01–srad_12`), average temperature (`tavg_01–tavg_12`), and vapour pressure (`vapr_01–vapr_12`) sampled from local GeoTIFF rasters.
- SoilGrids pH and soil organic carbon are fetchable through `src/soilgrids_data.py`, but those fields are currently commented out during dataset assembly and are not used by the trained model.

3.3 Cleaning and Dataset Size

`src/clean_data.py` converts the merged parquet to CSV and removes rows containing common no-data or overflow sentinels in any numeric column, including `-9999`, `-32768`, `32767`, `65535`, or values larger than `1e30` in magnitude. Table 1 summarizes the largest cleaned dataset currently present in the repository.

Table 1: Current local dataset used by the implemented training pipeline. The “raw merged” row refers to `combined_uniprot_and_gold_environmental_data_10K.csv`; the “cleaned” row refers to `cleaned_environmental_data_10K.csv`.

Dataset	Rows	Unique taxonomy IDs	Unique UniProt accessions	Median sequence length
Raw merged	10,113	830	8,888	381 aa
Cleaned	8,631	720	7,809	381 aa

In the cleaned 10K dataset, 8,376 rows have NASA temperature, and 8,296 rows have NASA radiation. Protein sequence lengths range from 6 to 11,872 amino acids, with a mean length of 450 amino acids. During training, sequences are truncated to 198 amino acids plus start and end tokens, so very long proteins are represented only by their N-terminal prefix.

4 Methods

4.1 Feature Engineering

The model uses three environmental input features:

1. `NASA_Temp_C`: daily NASA POWER temperature for the sample location/date.
2. `Total_Annual_Prec`: sum of the 12 WorldClim monthly precipitation values.
3. `Mean_Annual_Solar_Rad`: mean of the 12 WorldClim monthly solar-radiation values.

Although `src/train.py` also computes `Mean_Annual_Temp`, that field is not included in the current model input. The three selected numeric features are standardized with scikit-learn’s `StandardScaler`. The fitted scaler is saved as `checkpoints/environmental_scaler.pkl` and is reused by the Streamlit app.

4.2 Sequence Encoding

The output sequence is a character-level amino-acid sequence. The vocabulary has 23 tokens: the 20 standard amino acids plus `<PAD>`, `<SOS>`, and `<EOS>`. Each training sequence is truncated to a maximum length of 200 tokens after adding start and end markers. Unknown or nonstandard amino-acid characters are currently mapped to `<PAD>`; this is simple, but it means rare sequence characters are treated like padding rather than modeled explicitly.

4.3 Model Architecture

The implemented model in `src/model.py` is a conditional LSTM protein generator. The standardized three-dimensional environmental vector is passed through two learned linear layers. One initializes the LSTM hidden state and the other initializes the LSTM cell state. The amino-acid tokens are embedded into 64-dimensional vectors, processed by a single-layer LSTM with hidden size 256, and projected to the 23-token vocabulary at each position. During training, the model learns next-token prediction: given the environment and all previous tokens, it predicts the next amino acid.

4.4 Training and Evaluation Protocol

`src/train.py` drops rows missing sequence, NASA temperature, annual precipitation, or annual solar radiation. It then splits rows into 90% training and 10% validation using `random_state=42`. Training uses Adam with learning rate 0.001 for 100 epochs, batch size 32, and cross-entropy loss over next-token predictions. Padding tokens are ignored in the loss. The best model by validation loss is saved to the requested checkpoint path.

This split prevents exact row overlap between training and validation, but it is not a stringent biological generalization test. Multiple proteins from the same organism, taxonomy ID, or geographic location can appear on both sides of the split. Therefore, validation loss measures held-out row prediction under random splitting, not generalization to unseen species, protein families, or environments.

5 Results

5.1 End-to-End Pipeline

The main result is a working end-to-end pipeline. `src/main.py` builds a merged protein/environment table, `src/clean_data.py` removes corrupted numeric rows, `src/train.py`

trains and checkpoints the conditional LSTM, and `src/app.py` loads the checkpoint and scaler into an interactive Streamlit interface. The repository currently contains trained checkpoints for the 5K and 10K data runs, plus a fitted environmental scaler.

5.2 Sequence Space Visualization

`src/visualization.py` provides an exploratory check of whether the sequence dataset contains visible environmental structure. It converts each protein sequence into 3-mer TF-IDF features, reduces those features to two principal components, and colors the resulting points by environmental variables. This is not a model-evaluation metric, but it is useful for checking whether obvious climate gradients or data artifacts appear in sequence feature space.



Figure 1: Exploratory protein sequence space from the cleaned dataset. Each point is one protein sequence represented by 3-mer TF-IDF features and reduced to two principal components. Panels color the same points by NASA collection-day temperature, WorldClim annual temperature, WorldClim annual precipitation, WorldClim annual solar radiation, and NASA collection-day radiation.

5.3 Generation Interface

The trained model can generate a sequence from user-specified temperature, annual precipitation, and solar-radiation values. In command-line inference, `src/train.py` tests three example environments: an Earth-like temperate case, a hot arid high-radiation case, and a frozen low-radiation case. `src/app.py` exposes the same generation function in Streamlit with sliders and preset environments. Generation is greedy: at each step, the model chooses the most likely next token until it emits `<EOS>` or reaches the maximum generation length.

6 Discussion and Limitations

The project demonstrates that a heterogeneous genomics-and-environment dataset can be assembled and used to condition a sequence generator. The codebase now has modular data retrieval, cleaning, training, checkpointing, visualization, and UI components. This is the strongest outcome of the current work: the pipeline is reproducible and can be iterated on.

The biological interpretation is still limited. First, the input features are coarse descriptors of sampling location, not direct measurements of the microenvironment where every organism grew. Second, proteins are strongly shaped by function and ancestry; a random row split can overestimate performance if closely related organisms or similar protein families appear in both training and validation. Third, the model generates short prefixes of proteins because training truncates sequences to 200 tokens. Finally, the current code does not save final validation loss, per-residue accuracy, edit distance, or biological plausibility metrics to a results table. Those metrics should be added before making strong claims about model quality.

The next most important improvements are therefore methodological rather than architectural: save training curves and validation metrics, split data by taxonomy ID or protein family, add sequence-quality filters for nonstandard amino acids, compare against an unconditional LSTM baseline, and evaluate generated sequences with protein-level summaries such as amino-acid composition, hydrophobicity, and predicted stability. SoilGrids features are also already implemented and could be added to the model input once their coverage and units are checked.

7 Reproducibility

The main commands are:

```
python src/main.py

python src/clean_data.py \
  --input-parquet combined_uniprot_and_gold_environmental_data_10K.parquet \
  --output-csv combined_uniprot_and_gold_environmental_data_10K.csv \
  --cleaned-output-csv cleaned_environmental_data_10K.csv

python src/train.py \
  --data cleaned_environmental_data_10K.csv \
  --checkpoint checkpoints/best_alien_protein_model_10K.pt

streamlit run src/app.py -- \
  --checkpoint checkpoints/best_alien_protein_model_10K.pt \
  --scaler checkpoints/environmental_scaler.pkl
```

The WorldClim GeoTIFFs are expected under `data/worldclim/`. The main scripts are organized under `src/`; checkpoints and the scaler are stored under `checkpoints/`; exploratory figures are written to the repository root by `src/visualization.py`. External API calls require internet access and may change if source services update their schemas or availability.

References

- [1] GOLD: Genomes Online Database. <https://gold.jgi.doe.gov/>

- [2] The UniProt Consortium. UniProt: the Universal Protein Knowledgebase. <https://www.uniprot.org/>
- [3] NASA POWER Project. Prediction Of Worldwide Energy Resources. <https://power.larc.nasa.gov/>
- [4] WorldClim version 2.1 climate data. <https://www.worldclim.org/>
- [5] ISRIC SoilGrids. <https://soilgrids.org/>